

گزارش پروژه ریاضیات گسسته

بخش رمز جانشینی

MONOALPHABETIC SUBSTITUTION CIPHER

A	B	C	D	E	F	G
Q	M	Z	T	R	A	K

شماره دانشجویی

40420993

نویسنده

پارسا طباطبایی

بخش رمز جانشینی (Substitution)

این پوشه پیاده‌سازی رمز جانشینی تک‌الفبایی را شامل می‌شود. بخش‌های مربوط به رمزگذاری، رمزگشایی، تحلیل آماری، آزمایش‌ها و تست‌ها در فایل‌ها و پوشه‌های جدا قرار گرفته‌اند.

در این پیاده‌سازی پاک‌سازی فقط برای محاسبات آماری انجام می‌شود. هنگام رمزگذاری، جای حروف، فاصله‌ها، اعداد و نشانه‌هایی مانند نقطه و ویرگول ثبت می‌ماند و فقط خود حروف جانشین می‌شوند. بزرگی و کوچکی حروف انگلیسی نیز حفظ می‌شود؛ بنابراین رمزگذاری و سپس رمزگشایی `Hello.World!` دوباره دقیقاً `Hello.World!` را می‌سازد. برای فارسی شکل عربی `ك` و `ي` هنگام پردازش به `ک` و `ی` استاندارد می‌شود.

روش پیاده‌سازی

ساختار کد ساده و مرحله‌ای است:

- کلید یک دیکشنری معمولی از «حرف متن اصلی» به «حرف متن رمز» است.
- رمزگشایی با معکوس کردن همین دیکشنری انجام می‌شود.
- حمله آماری همان روش ساده مرتب‌کردن حروف برحسب فراوانی است.
- آمار روی حروف پاک‌شده محاسبه می‌شود، اما خروجی با قالب متن ورودی بازسازی می‌شود.
- نمودار و اجرای آزمایش از منطق رمز جدا شده‌اند و دیگر با `import` کردن فایل نمایش داده نمی‌شوند.

فایل‌های اصلی پروژه در پوشه `src` قرار دارند. منطق رمز در `substitution_cipher.py`، محاسبات آماری در `text_statistics.py`، رمزشکنی در `substitution_cryptanalysis.py` و آزمایش‌ها در `experiment.py` نوشته شده‌اند.

ساختار فایل‌ها

- `src/substitution_cipher.py`: پاک‌سازی متن، تولید کلید، رمزگذاری و رمزگشایی
- `src/substitution_cryptanalysis.py`: جدول مرجع انگلیسی، حمله فراوانی و محاسبه دقت
- `src/text_statistics.py`: فراوانی حرف، دوگرام، سه‌گرام و مقایسه آنتروپی شانون
- `src/experiment.py`: آزمایش دقت حمله نسبت به طول متن و ذخیره CSV/PNG
- `main.py`: منوی تعاملی و رابط خط فرمان
- `tests/test_substitution.py`: تست‌های خودکار
- `data/sample_plaintext.txt`: پیکره نمونه انگلیسی برای آزمایش
- `data/sample_plaintext_fa.txt`: پیکره نمونه فارسی برای آزمایش
- `data/english_reference.txt`: پیکره مستقل برای مدل bigram/trigram انگلیسی
- `data/persian_reference.txt`: پیکره مستقل برای مدل bigram/trigram فارسی
- `output/`: محل جداگانه نمودارها و گزارش‌های تولیدشده

مدلسازی ریاضی و اثبات‌ها

۱. الفبا و تعریف کلید

فرض کنید الفبا مجموعه متناهی زیر با q عضو باشد:

```
A = {a_0, a_1, ..., a_{(q-1)}}
```

برای انگلیسی $q = 26$ و در الفبای فارسی تعریف‌شده در برنامه $q = 32$ است. کلید جانشینی تابع زیر است:

```
 $\pi: A \rightarrow A$ 
```

کلید باید یک جایگشت یا تابع دوسویی باشد؛ یعنی هر حرف دقیقاً به یک حرف رمز متفاوت نگاشته شود و همه حروف مقصد دقیقاً یک‌بار استفاده شوند. تابع `generate_random_key` با برزیدن الفبا چنین جایگشتی می‌سازد و `key_from_string` نیز کامل و یکتا بودن کلید ورودی را کنترل می‌کند.

۲. فرمول رمزگذاری و رمزگشایی

اگر دنباله حروف قابل رمزگذاری متن $P = P_0 P_1 \dots P_{(n-1)}$ باشد، هر حرف مستقل از موقعیت خود جایگزین می‌شود:

```
Encryption:  $C_i = \pi(P_i)$   
Decryption:  $P'_i = \pi^{-1}(C_i)$ 
```

در این روش جایگاه i در انتخاب حرف جانشین نقشی ندارد و کلید نیز دوره‌ای نیست؛ یک حرف مشخص در تمام متن همیشه به یک حرف مشخص تبدیل می‌شود.

اثبات درستی رمزگشایی

چون π یک تابع دوسویی روی مجموعه متناهی A است، تابع معکوس π^{-1} وجود دارد. فرمول رمزگذاری را در رمزگشایی قرار می‌دهیم:

```

P'_i = π-1(C_i)
      = π-1(π(P_i))
      = P_i

```

این تساوی برای همه i ها برقرار است، بنابراین:

```

Decrypt(Encrypt(P, π), π) = P

```

تست‌های رفت‌وبرگشت انگلیسی و فارسی همین نتیجه را بررسی می‌کنند. نشانه‌ها و کاراکترهای خارج از الفبای انتخاب‌شده تحت تابع همانی قرار می‌گیرند، یعنی $\pi(x) = x$ ؛ در نتیجه آن‌ها نیز بدون تغییر و در همان اندیس باقی می‌مانند. در انگلیسی وضعیت uppercase هر جایگاه پس از جانشینی دوباره اعمال می‌شود.

● ۳. تعداد کلیدها و ناممکن بودن جست‌وجوی کامل

برای تصویر اولین حرف q انتخاب وجود دارد. پس از انتخاب آن، برای حرف دوم $q - 1$ انتخاب، برای حرف سوم $q - 2$ انتخاب و در پایان یک انتخاب باقی می‌ماند. طبق اصل ضرب:

```

|K| = q × (q - 1) × ... × 2 × 1 = q!

```

در نتیجه:

```

English: 26! = 403291461126605635584000000 ≈ 4.03 × 1026
Persian: 32! ≈ 2.63 × 1035

```

اگر حتی یک میلیارد کلید در ثانیه بررسی شود، پیمایش $26!$ کلید انگلیسی بیش از ۱۲/۷ میلیارد سال طول می‌کشد. علاوه بر آن، آزمایش هر کلید روی متن n حرفی به $O(n)$ زمان نیاز دارد؛ پس مرتبه زمانی brute force برابر است با:

```

O(n × q!)

```

به همین دلیل برنامه از اطلاعات آماری زبان استفاده می‌کند و همه جایگشت‌ها را امتحان نمی‌کند.

۴. اثبات حفظ شدن فراوانی حروف

تعداد رخداد حرف a در متن اصلی را با $f_P(a)$ نشان می‌دهیم. هر بار که a دیده شود، رمزگذاری دقیقاً $\pi(a)$ تولید می‌کند. همچنین چون کلید دوسویی است، هیچ حرف دیگری به $\pi(a)$ تبدیل نمی‌شود. پس یک تناظر یک‌به‌یک میان رخدادها و این دو حرف برقرار است و داریم:

$$f_C(\pi(a)) = f_P(a)$$

اگر طول متن N باشد، فراوانی نسبی یا احتمال تجربی حرف برابر است با:

$$p_P(a) = f_P(a) / N$$

$$p_C(\pi(a)) = f_C(\pi(a)) / N = p_P(a)$$

پس رمز جانشینی مقدارهای توزیع فراوانی را تغییر نمی‌دهد و فقط برچسب حروف را جابه‌جا می‌کند. این ویژگی هم دلیل امکان حمله فراوانی و هم ضعف اصلی رمز جانشینی تک‌الفبایی است.

۵. روش حمله فراوانی پیاده‌سازی شده

فرض کنید حروف متن رمز پس از شمارش به ترتیب زیر قرار بگیرند:

$c_1, c_2, \dots, c_r$

و حروف جدول مرجع زبان نیز از پرتکرارترین به کم‌تکرارترین چنین باشند:

$e_1, e_2, \dots, e_q$

الگوریتم ساده پروژۀ نگاشت رمزگشایی حدسی زیر را می‌سازد:

$g(c_i) = e_i \quad \text{for } i = 1, \dots, \min(r, q)$

برای انگلیسی ابتدای جدول مرجع $(e, t, a, o, i, n, \dots)$ است. پس پرتکرارترین نماد رمز به e ، نماد بعدی به t و به همین ترتیب نگاشت می‌شود. جدول فارسی نیز از شمارش حروف در نمونه‌ای شامل ۱۰٪ مقالات ویکی‌پدیای فارسی ساخته شده و با $(\dots, \text{ء, ی, ر, د, ن})$ آغاز می‌شود. شمارش $\bar{آ}$ در این پروژۀ با $\bar{ا}$ ادغام شده تا جدول با الفبای ۳۲

حرفی کلید یکسان باشد. داده و روش شمارش حروف فارسی

پس از ساخت کلید حدسی، برنامه فقط جایگاه‌های حروف را جایگزین می‌کند. فاصله، نقطه، ویرگول، عدد و سایر نشانه‌ها مستقیماً از متن رمز به خروجی منتقل می‌شوند و بزرگی حروف انگلیسی نیز دوباره روی حرف حدس‌زده شده اعمال می‌شود.

این روش یک تخمین است و اثباتی وجود ندارد که در هر متن محدود رتبه حروف دقیقاً با جدول مرجع برابر باشد. موضوع متن، سبک نویسندگی و کوتاه بودن نمونه می‌تواند رتبه‌ها را عوض کند. در نسخه فعلی، این نداشت فقط نقطه شروع است و بخش n-gram آن را با جابه‌جایی دو مقدار کلید و مقایسه کیفیت متن‌های حاصل اصلاح می‌کند.

● ۶. چرا طول متن روی نتیجه اثر دارد؟

اگر احتمال واقعی یک حرف در زبان $p(a)$ باشد، فراوانی نسبی مشاهده شده در یک نمونه N حرفی چنین است:

```
p_hat_N(a) = f_N(a) / N
```

طبق قانون اعداد بزرگ، با افزایش N این مقدار معمولاً به $p(a)$ نزدیک می‌شود:

```
p_hat_N(a) -> p(a) as N -> infinity
```

بنابراین در متن‌های بلندتر رتبه تجربی حروف معمولاً پایدارتر است و حمله شانس بیشتری دارد. این یک روند احتمالی است، نه تضمین افزایش یکنواخت دقت در تک‌تک نمونه‌ها؛ ممکن است دقت در دو طول متوالی موقتاً کاهش پیدا کند.

● بررسی n-gramها

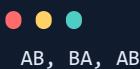
علاوه بر تک‌حرف‌ها، بخش **Analyze** دوگرام‌ها و سه‌گرام‌های متن پاک‌شده را نیز می‌شمارد. اگر دنباله حروف پاک‌شده $X_0, X_1, \dots, X_{(N-1)}$ باشد، n-gram شروع شده از جایگاه i برابر است با:

```
G_i = X_i X_{(i+1)} ... X_{(i+n-1)}
```

در یک متن N حرفی، تعداد کل n-gramهای هم‌پوشان برابر است با:

```
N - n + 1
```

مثلاً متن پاک‌شده **ABAB** دارای این دوگرام‌هاست:



AB, BA, AB

پس شمارش **AB** برابر ۲ و شمارش **BA** برابر ۱ است. فراوانی نسبی هر n-gram نیز از تقسیم شمارش آن بر تعداد کل n-gramها محاسبه می‌شود. گزارش JSON همهٔ شمارش‌ها و درصدها را نگه می‌دارد و نمودار PNG بیست دوگرام و بیست سه‌گرام پرتکرار را نمایش می‌دهد.

تابع **Break** از پیکرهٔ مستقل `data/english_reference.txt` احتمال لگاریتمی bigramها و trigramها را می‌سازد. برای یک متن کاندید، امتیاز زیر محاسبه می‌شود:



```
score(P) = sum(log Pr(bigram)) + 1.35 × sum(log Pr(trigram))
```

کلید حدسی فراوانی نقطهٔ شروع است. در هر iteration دو مقدار نگاشت کلید جابه‌جا می‌شوند؛ اگر متن حاصل امتیاز n-gram بهتری داشته باشد، تغییر نگه داشته می‌شود. پذیرش احتمالی بعضی تغییرهای ضعیف‌تر و چند restart کمک می‌کند جست‌وجو در یک جواب محلی متوقف نشود. پیکرهٔ مرجع فقط الگوی عمومی زبان را فراهم می‌کند و متن اصلی آزمایش به تابع Break داده نمی‌شود.

● ۷. آنتروپی شانون زیر جانشینی

برای توزیع حروف، آنتروپی شانون برابر است با:



$$H(P) = -\sum_{a \in A} p_P(a) \times \log_2(p_P(a))$$

چون در بخش قبل ثابت شد $p_C(\pi(a)) = p_P(a)$ و π فقط ترتیب جمله‌های مجموع را جابه‌جا می‌کند:



$$\begin{aligned} H(C) &= -\sum_{a \in A} p_C(\pi(a)) \times \log_2(p_C(\pi(a))) \\ &= -\sum_{a \in A} p_P(a) \times \log_2(p_P(a)) \\ &= H(P) \end{aligned}$$

پس یک جانشینی تک‌الفبایی ایده‌آل آنتروپی تک‌حرفی را افزایش نمی‌دهد. این نتیجه با حفظ فراوانی سازگار است و نشان می‌دهد چرا نمودار حروف متن اصلی و متن رمز شکل یکسانی با برجسب‌های متفاوت دارد.

برنامه برای هر دو متن تعداد حروف، آنتروپی برجسب بیت بر حرف و درصد نسبت به بیشینهٔ $\log_2(q)$ را محاسبه می‌کند. اختلاف آنتروپی نیز در `output/entropy_comparison.json` ذخیره می‌شود. برای انگلیسی بیشینه $\log_2(26) \approx 4.7004$ و برای فارسی $\log_2(32) = 5$ بیت بر حرف است.

● ۸. فرمول دقت و مدل آزمایش

اگر متن اصلی مورد مقایسه P و خروجی حدسی G هر دو N کاراکتر داشته باشند، دقت کاراکتر به کاراکتر برابر است با:

$$\text{accuracy}(P, G) = (1 / N) \times \sum_{i=0 \text{ to } N-1} I(P_i = G_i) \times 100$$

تابع شاخص I در صورت برابری دو حرف مقدار ۱ و در غیر این صورت مقدار صفر دارد. برای هر طول L ، آزمایش به تعداد T بار یک بخش تصادفی از پیکره و یک کلید تصادفی جدید انتخاب می‌کند، متن را رمزگذاری و بدون کلید بازیابی می‌کند. سپس میانگین دقت حرفی و نرخ بازیابی کاملاً صحیح محاسبه می‌شوند:

$$\begin{aligned} \text{average_accuracy}(L) &= (\text{accuracy}_1 + \dots + \text{accuracy}_T) / T \\ \text{exact_success_rate}(L) &= \text{exact_successes} / T \\ \text{exact_key_rate}(L) &= \text{exactly_recovered_keys} / T \end{aligned}$$

سه مقدار در CSV و نمودار PNG ذخیره می‌شوند: میانگین دقت حرفی، نرخ بازیابی کامل متن و نرخ بازیابی کامل کلید. برای معیار سوم، تمام ۲۶ نگاشت حدس زده شده باید با معکوس کلید واقعی یکسان باشند. seed پیش فرض 1405 باعث می‌شود کل نمونه‌گیری و کلیدهای تصادفی دوباره قابل تولید باشند.

● ۹. پیچیدگی زمانی

- پاک‌سازی، رمزگذاری و رمزگشایی متن n حرفی: $O(n)$
- ساخت یا معکوس کردن کلید با الفبای q حرفی: $O(q)$
- شمارش فراوانی: $O(n)$
- مرتب‌سازی حروف برحسب فراوانی: $O(q \log q)$
- کل حمله رتبه‌ای: $O(n + q \log q)$

چون اندازه الفبا ثابت و کوچک است، هزینه عملی حمله تقریباً خطی نسبت به طول متن است؛ این مقدار بسیار کمتر از $O(n \times q!)$ جست‌وجوی کامل است.

خلاصه روش رمز شکنی

1. یک نسخه فقطحرفی از متن رمز برای محاسبه آمار ساخته می‌شود؛ متن اصلی نیز برای بازسازی نگه داشته می‌شود.
2. تعداد و درصد رخداد هر حرف محاسبه می‌شود.
3. حروف رمز از پرتکرارترین تا کم‌تکرارترین مرتب می‌شوند.
4. این ترتیب با جدول استاندارد زبان تطبیق داده می‌شود.
5. با نگاشت حدسی «حرف رمز به حرف اصلی»، حروف جایگزین و قالب، نشانه‌ها و uppercase متن بازسازی می‌شوند.
6. اگر متن اصلی آزمایش موجود باشد، دقت حرف‌به‌حرف محاسبه می‌شود.

اجرا

دستورها را داخل پوشه `substitution` اجرا کنید. برای منوی ساده تعاملی:

```
python main.py
```

در حالت تعاملی، برای رمزگذاری، رمزگشایی، شکستن رمز و تحلیل می‌توان متن را مستقیم تایپ کرد یا از فایل UTF-8 خواند. مسیر فایل خروجی نیز قابل انتخاب است. در گزینه **6** برنامه زبان، فایل پیکره، طول‌های آزمایش، تعداد تکرار برای هر طول، seed و پوشه خروجی را می‌پرسد. با فشردن Enter مقدار پیش‌فرض داخل کروشه انتخاب می‌شود.

تولید کلید تصادفی انگلیسی (خروجی یک جایگشت ۲۶ حرفی است):

```
python main.py generate-key --language en --seed 1405
```

در رشته کلید، حرف اول جانشین **a**، حرف دوم جانشین **b** و به همین ترتیب است. نمونه با کلید معکوس الفبا:

```
python main.py encrypt --text "Hello world!" --key zyxwvutsrqponmlkjihgfedcba
python main.py decrypt --text "Svool dliow!" --key zyxwvutsrqponmlkjihgfedcba
```

شکستن متن انگلیسی با جدول فراوانی، بدون داشتن کلید:

```
python main.py break --language en --algorithm frequency --input output/ciphertext.txt --output
output/recovered.txt
python main.py break --language en --algorithm hill-climbing --input output/ciphertext.txt --restarts 3
--iterations 2000 --seed 1405 --output output/recovered_hill.txt
python main.py break --language en --algorithm simulated-annealing --input output/ciphertext.txt --
restarts 3 --iterations 2000 --seed 1405 --output output/recovered_annealing.txt
```

شکستن متن فارسی بدون کلید با جدول فراوانی فارسی:

```
python main.py break --language fa --algorithm simulated-annealing --input
output/persian_ciphertext.txt --output output/persian_recovered.txt
```

ساخت گزارش JSON و نمودار سه‌بخشی حرف، bigram و trigram:



```
python main.py analyze --input data/sample_plaintext.txt --label plaintext --output-dir output
```

مقایسه آنتروپی شانون plaintext و ciphertext:



```
python main.py compare-entropy --language en --plaintext-input data/sample_plaintext.txt --ciphertext-input output/ciphertext.txt --output output/entropy_comparison.json
```

در منوی تعاملی نیز گزینه 7 همین مقایسه را انجام می‌دهد و هر دو متن را مستقیم یا از فایل UTF-8 دریافت می‌کند.

رسم نمودار دقت حمله برحسب طول متن و ذخیره داده‌های نمودار در CSV:



```
python main.py experiment --language en --corpus data/sample_plaintext.txt --reference-corpus data/english_reference.txt --lengths 200 500 1200 3000 5000 8000 --trials 20 --ngram-restarts 3 --ngram-iterations 1500 --seed 1405 --output-dir output
```

برای نمودارها ابتدا وابستگی را نصب کنید:



```
python -m pip install -r requirements.txt
```

اجرای تست‌ها:



```
python -m unittest discover -s tests -v
```

نکات تحلیلی برای گزارش

- کلید معتبر باید یک جایگشت کامل باشد؛ کلید ناقص یا دارای حرف تکراری معکوس یکتا ندارد.
- رمز جانشینی تعداد رخداد هر حرف و آنتروپی تک حرفی را حفظ می‌کند.
- جدول فراوانی مرجع میانگین زبان است و الزاماً با یک متن خاص برابر نیست.
- بخش Analyze شمارش و نمودار n-gram را تولید می‌کند و بخش Break از مدل n-gram برای اصلاح عملی کلید فراوانی استفاده می‌کند.
- افزایش `ngram-iterations` و `ngram-restarts` معمولاً شانس فرار از جواب محلی را بیشتر می‌کند، اما زمان اجرا نیز افزایش می‌یابد.
- نزدیک بودن فراوانی بعضی حروف باعث جابه‌جایی رتبه و خطای حمله ساده می‌شود.
- فاصله‌ها، اعداد، نشانه‌ها و بزرگی حروف انگلیسی در رمزگذاری، رمزگشایی و خروجی حمله حفظ می‌شوند.
- پاک‌سازی فقط برای شمارش فراوانی استفاده می‌شود و قالب متن اصلی را از بین نمی‌برد.
- شکستن بدون کلید از هر دو جدول مرجع انگلیسی و فارسی پشتیبانی می‌کند؛ کافی است `--language en` یا `-language fa` انتخاب شود.
- این رمز و این حمله صرفاً آموزشی‌اند و برای امنیت واقعی مناسب نیستند.

پایان گزارش بخش رمز جانشینی